

UNION-FIND

Struktura - los drzew:

Find(x) - przechodzący
ścieżkę od x do korzenia,
potem kompresję ścieżki

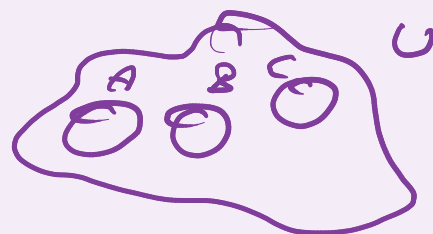
UNION(A,B) - sposób dołączający

> m - ciąg operacji Union i Find

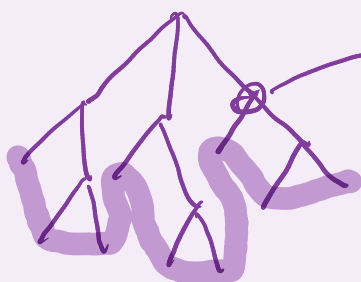
> n - moc uniwर्सum U

> ilość operacji union $\leq n-1$ (dla prostoty oznaczamy przez n)

> ilość operacji find - m



red(v)



→ red - długość ścieżki (krawędziowej) do liścia

→ red 0 dla liści

$\forall x$ na ścieżce od v do korzenia red węzłków jest rosnący

FAKT Węzłków redu r jest $\leq \frac{n}{2^r}$

Def (1) $F(i) = \begin{cases} 1 & i=0 \\ 2^{F(i-1)} & \text{dla } i>0 \end{cases}$ Czyli: $F(i) = i!$

(2) $\log^* n = \min(k: F(k) \geq n)$

Grupy rzędów:

rzęd r jest w grupie $\log^* r$

Grupa	rzędy w tej grupie
0	0, 1
1	2
2	3, 4
3	5, 6, ..., 16
4	17, 18, ..., 2^{16}
...	...
i	$F(i-1)+1, \dots, F(i)$

UWAGA:

• $F(n)$ to po prostu n -ta tetracja liczby 2, czyli:

$$F(n) = {}^n 2 = \underbrace{2^{2^{\dots^2}}}_{n \text{ dażeń}}$$

• $\log^* n$ to po prostu podloga z odwróceniem tej tetracji

Analiza kosztu wykonania $\mathcal{O}$:

$$\mathcal{O}: F_1 F_2 U_1 F_3 F_4 U_2 \dots$$

$\text{Koszt}(\mathcal{O}) = \text{koszt wszystkich unionów} + \text{koszt wszystkich findów}$

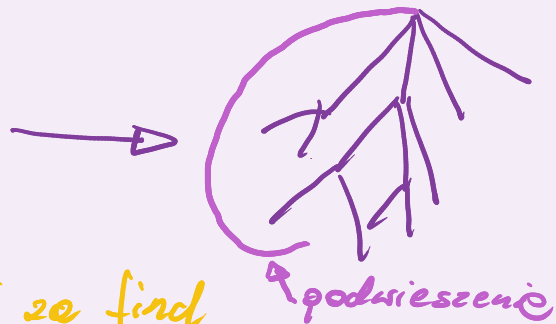
Koszt wszystkich unionów jest ograniczony przez n .

Koszt pojedynczego uniona to $O(1)$ zatem koszt wszystkich to $O(n)$.

Koszt pojedynczego $\text{Find}(u)$ - długość ścieżki z u do korzenia.



 $\rightarrow$ wiezchołek płaci za find



Niestety naiwne przypisywanie kosztu każdemu węzłowi jest niemięrodojne (tzn. przypisywanie tylu monet ile wynosi długość ścieżki do korzenia).

Koszt pojedynczego $\text{Find}(v)$ rozdzielamy między

→ instrukcję Find

↳ gdy:

- ✓ odwiedźmy korzeń
- ✓ odwiedźmy syna korzenia
- ✓ odwiedźmy, tj. v oraz $\text{ojciec}(v)$ może być w innych grupach

→ odwiedzone wierzchołki:

↳ w pozostałych przypadkach (tj. w tych gdzie nie odwiedźmy find).

Wystarczy, aby wszystko opłacić, ponieważ płacimy za każdy etap każdej wędrówki, po prostu rozdzielamy wszystkie monety pomiędzy wierzchołki, a find y, jedyne co potrzebujemy opłacić to ile monetka naprawdę będzie potrzebne.

Koszt wszystkich operacji find możemy oznaczyć jako:

$$\underbrace{\sum_{F \in \mathcal{F}} \text{Obciążenie}(F)}_{(1)} + \underbrace{\sum_{v \in V} \text{Obciążenie}(v)}_{(2)}$$

(1) Obciążenie pojedynczego $\text{find} \leq 2 +$

↳
za odwiedzenie
korzenia i jego syna

↑
liczba zmiennych
grup wędrówek
w trakcie wędrówki
do korzenia,
ale takich zmiennych jest
nie więcej niż liczba
grup - 1.

maksymalna głębokość wierzchołka $\leq \log n$

maksymalny numer grupy $\leq \log^*(\log n)$

Zauważmy, że $\log^* n$ wyznacza ilość logarytmowań n jako wykonamy dopóki nie dojdziemy do 1

$$\log_2 \underbrace{2^{2^{\dots^2}}}_{k+1 \text{ dwójek}} = c, \text{ że } 2^c = \underbrace{2^{2^{\dots^2}}}_{k \text{ dwójek}} \rightarrow \text{to jest } c \rightarrow c = \underbrace{2^{2^{\dots^2}}}_{k-1 \text{ dwójek}}$$

$\Rightarrow$ pojedynczy log zmniejsza nam $\log$ o 1.

Stąd $\log^*(\log n) = \log^* n - 1$

Stąd obciążenie wszystkich operacji $FMD \leq m \cdot \log^* n$

Szacujemy obciążenie wszystkich wierzchołków

$\forall$ grupy g z grupy g Policzmy sumaryczne obciążenie o zędach

Szacujemy ilość wierzchołków o zędach w grupie g .

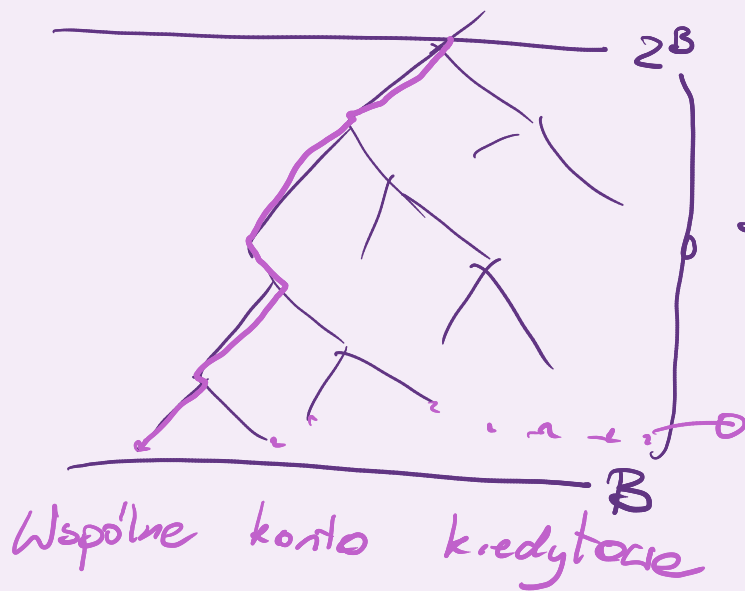
$$\sum_{r \in \text{grupy } g} \text{liczba wierzchołków o rzędzie } r = F(g)$$

$$= \sum_{r=F(g-1)+1} \text{liczba wierzchołków o rzędzie } r =$$

$$\leq \sum_{r=F(g-1)+1}^{F(g)} \frac{n}{2^r} = \frac{n}{2^{F(g-1)+1}} + \frac{n}{2^{F(g-1)+2}} + \dots + \frac{n}{2^{F(g)}} <$$

$$\frac{n}{2^{F(g-1)+1}} \left(1 + \frac{1}{2} + \frac{1}{4} + \dots \right) = \cancel{2} \cdot \frac{n}{2^{F(g-1)+1}} = \frac{n}{2^{F(g-1)}} \stackrel{\text{zdefiniuj } F}{=} \frac{n}{F(g)}$$

Każdy wierzchołek o zędzie z grupy g ma być obciążony nie więcej razy niż główna liczba zędów w grupie g , czyli $\leq F(g) - F(g-1)$



sumaryczne obciążenie $\leq O(n)$

punkty startowe $\leq \frac{n}{2^B}$ węzłów
 z każdego punktu startowego nie dłuższa ścieżka, niż $|2^B - B|$

DLACZEGO?

$F(g-1)+1$



Idea: w górę do korzenia przemierzamy wierzchołki o ściśle wyższych rzędach - stąd z każdego wierzchołka zmieniamy rząd o -1 .

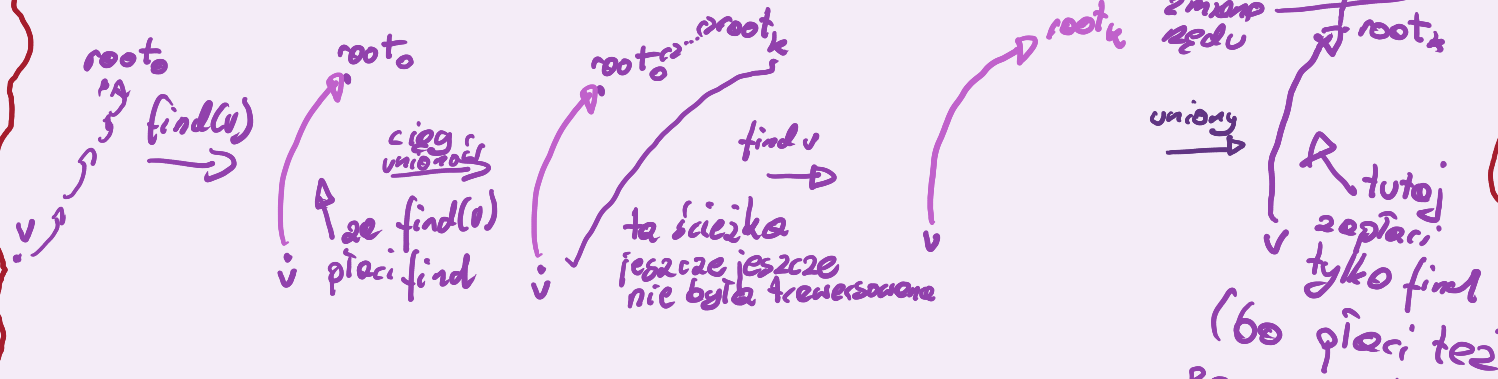
Stąd niezgłuszenie w worst-case przemierzamy wszystkie rzędy z g i obcinamy ∇ niepotrzebne wierzchołki.

Stąd sumaryczne obciążenie wierzchołków o rzędach z grupy g jest $< \underbrace{\frac{n}{F(g)}}_{\text{górne ograniczenie na ilość wierzchołków z grupy } g} \underbrace{(F(g) - F(g-1))}_{\text{ile razy obcinamy każdy wierzchołek}} \leq \frac{n}{F(g)} \cdot F(g) = n$

górne ograniczenie na ilość wierzchołków z grupy g

ile razy obcinamy każdy wierzchołek

Jest tak dlatego, że nie stawiamy tej samej ścieżki z v dwukrotnie (bo mamy kompresję ścieżek) oraz część operacji opłaca tylko find.



Jeśli koreń był w innym rzędzie, to kolejne find(v) będą nas automatycznie przenosić nas do wyższych rzędów.

Stąd ograniczenie wszystkich wierzchołków:

$$\sum_g (\text{sumaryczne ograniczenie wierzchołków o stopniu } \geq g) \leq \text{ilość grup} \cdot n = O(n \log^* n).$$

Po zsumowaniu ograniczeń findów i wierzchołków otrzymujemy $(n+m) \log^* n$.

Ich habe genug! 

Drzewa splay

Idea $\rightarrow$ wykonanie operacji słownikowej na wierzchołku v powoduje przesunięcie v do korzenia.

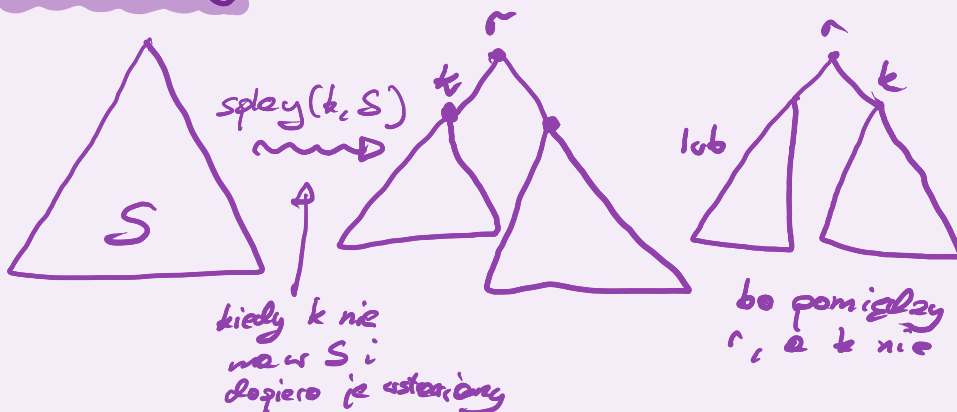
Operacja $\text{splay}(v, S)$ - $\uparrow$ drzewo $\sim$ przeorganizuj S tak by v korzeniem znalazł się:
 $\left\{ \begin{array}{l} v \text{ jeśli } v \text{ jest w } S \\ u, u \in S \text{ i pomagający } u, \text{ a } v \text{ nie ma innego domownika w } S \end{array} \right.$

Realizacja $\text{splay}(v, S)$:

- najłatwiej - rotacjami windyujemy v do korzenia (słabe)

Skomplikowanie - każdą operację słownikową można zaimplementować tak, by wykonana była stałą liczbą operacji splay i stałą liczbą operacji niskiego rzędu.

Wstawianie



Usuwanie

